```
"""
anomaly.py  v2
--------------
アノマリー検出レイヤー。v2の改善点：

【インフレ対策】
- 動的ベースライン：スコアを「現世代の集団中央値」からの相対値として計算する。
  全員が同じ方向に進化しても、集団内の相対的な逸脱だけをアノマリーと見なす。
  → 全員がアノマリーになることを構造的に防ぐ。

- 第二の正常（new normal）の検出：過去N世代のスコア分布を追跡し、
  「以前のアノマリーが今の正常になった」かどうかを記録する。

軸1：語彙乖離　- TF-IDFベクトルのコサイン距離（動的ベースライン補正付き）
軸2：構造乖離　- 文長・句読点・繰り返しの統計的逸脱
軸3：新語スコア　- 直近世代の集団語彙に存在しない語の割合

スコアは [0, 1]。ただし集団の中央値が常に ~0.5 に正規化されるため、
「全員がアノマリー」という状態は原理的に発生しない。
"""

import re
import numpy as np
from collections import Counter, deque
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_distances


class AnomalyDetector:
    def __init__(self, history_window: int = 10):
        self.vocabulary_history: list[str] = []
        self.history_window = history_window
        self.score_history: deque = deque(maxlen=history_window)
        self.generation = 0

    def score_population(self, texts: list[str]) -> list[float]:
        n = len(texts)
        if n < 2:
            return [0.0] * n

        scores_vocab   = self._vocab_divergence(texts)
        scores_struct  = self._structural_divergence(texts)
        scores_novelty = self._novelty_score(texts)
```

```
        raw = np.array([
            0.40 * scores_vocab[i]
            + 0.30 * scores_struct[i]
            + 0.30 * scores_novelty[i]
            for i in range(n)
        ])

        # 動的ベースライン正規化（インフレ対策）
        normalized = self._dynamic_normalize(raw)

        self.vocabulary_history.extend(texts)
        if len(self.vocabulary_history) > 500:
            self.vocabulary_history = self.vocabulary_history[-500:]

        self.score_history.append(normalized.copy())
        self.generation += 1

        return [float(np.clip(s, 0.0, 1.0)) for s in normalized]

    def new_normal_threshold(self) -> float:
        """過去N世代の上位25%平均。下がってきたら第二の正常のサイン。"""
        if not self.score_history:
            return 0.5
        all_scores = np.concatenate(list(self.score_history))
        return float(np.percentile(all_scores, 75))

    def population_drift(self) -> float:
        """集団全体のドリフト量。大きいほど集団ごとアノマリー化が進んでいる。"""
        if len(self.score_history) < 2:
            return 0.0
        recent = np.mean(list(self.score_history)[-3:])
        early  = np.mean(list(self.score_history)[:3])
        return float(recent - early)

    def _dynamic_normalize(self, raw: np.ndarray) -> np.ndarray:
        """集団の中央値を0.5に固定して再スケール。全員高スコアを構造的に防ぐ。"""
        median = np.median(raw)
        mad    = np.median(np.abs(raw - median)) + 1e-9
        z = (raw - median) / (mad * 1.4826)
        return 1 / (1 + np.exp(-z * 0.8))

    def _vocab_divergence(self, texts: list[str]) -> list[float]:
        try:
            fit_corpus = self.vocabulary_history[-200:] + texts if self.vocabular
            vec = TfidfVectorizer(min_df=1, token_pattern=r"(?u)\b\w+\b", max_fea
            vec.fit(fit_corpus)
            matrix   = vec.transform(texts).toarray()
```

```
            centroid = matrix.mean(axis=0, keepdims=True)
            if centroid.sum() == 0:
                return [0.0] * len(texts)
            distances = cosine_distances(matrix, centroid).flatten()
            d_max = distances.max()
            if d_max == 0:
                return [0.0] * len(texts)
            return (distances / d_max).tolist()
        except Exception:
            return [0.0] * len(texts)

    def _structural_divergence(self, texts: list[str]) -> list[float]:
        features = []
        for t in texts:
            words  = t.split()
            length = len(words)
            punct_count  = sum(1 for c in t if c in ".,;:!?—…()[]\"'")
            punct_ratio  = punct_count / max(len(t), 1)
            counts       = Counter(w.lower() for w in words)
            repeat_ratio = sum(v for v in counts.values() if v > 1) / max(length,
            sents        = re.split(r"[.!?]", t)
            frag_score   = len([s for s in sents if s.strip()]) / max(length, 1)
            features.append([length, punct_ratio, repeat_ratio, frag_score])

        features = np.array(features, dtype=float)
        if features.shape[0] < 2:
            return [0.0] * len(texts)
        mean     = features.mean(axis=0)
        std      = features.std(axis=0) + 1e-9
        z_scores = np.abs((features - mean) / std).mean(axis=1)
        return (1 / (1 + np.exp(-(z_scores - 1.0)))).tolist()

    def _novelty_score(self, texts: list[str]) -> list[float]:
        # 直近50テキストだけ参照（古い語彙を忘れる→新語が定着したら普通になる）
        recent_pool = self.vocabulary_history[-50:] if self.vocabulary_history el
        if not recent_pool:
            return [0.0] * len(texts)
        pool_words = set()
        for t in recent_pool:
            pool_words.update(w.lower() for w in t.split())
        scores = []
        for t in texts:
            words = [w.lower() for w in t.split()]
            if not words:
                scores.append(0.0)
                continue
            novel = sum(1 for w in words if w not in pool_words)
```

```
            scores.append(novel / len(words))
        return scores

    def explain(self, text: str, texts: list[str]) -> dict:
        idx = texts.index(text) if text in texts else 0
        v = self._vocab_divergence(texts)
        s = self._structural_divergence(texts)
        n = self._novelty_score(texts)
        raw        = np.array([0.4*v[i] + 0.3*s[i] + 0.3*n[i] for i in range(len(
        normalized = self._dynamic_normalize(raw)
        return {
            "vocab_divergence":     round(v[idx], 4),
            "structural_diverge":   round(s[idx], 4),
            "novelty_score":        round(n[idx], 4),
            "raw_combined":         round(float(raw[idx]), 4),
            "normalized_score":     round(float(normalized[idx]), 4),
            "new_normal_threshold": round(self.new_normal_threshold(), 4),
            "population_drift":     round(self.population_drift(), 4),
        }
```